

Entregable — Control de Calidad (QA)

Integrante: Jhon Escobar **Rol:** Ingeniero de Control de Calidad (QA) **Proyecto:** Muelle Digital

1. Estrategia de pruebas

El software maneja **datos personales y pagos**, por lo que la calidad y la seguridad son prioritarias.

- **Pruebas funcionales:** verificar que cada flujo (registro, login, reserva, pago, reseña, verificación de guía) haga lo esperado.
- **Pruebas de seguridad:** encriptación de contraseñas, control de acceso por rol, protección de datos ajenos, claves fuera del repositorio.
- **Pruebas de regresión:** repetir el set principal tras cada cambio importante.
- **Niveles:** prueba de API (backend) + prueba de interfaz (frontend) + prueba de integración (extremo a extremo).
- **Herramientas:** Postman / curl (API), navegador + DevTools (UI), revisión de código vía Pull Requests de GitHub.

2. Casos de prueba detallados

ID	Caso	Pasos	Resultado esperado	Estado
TC-01	Registro válido	Enviar nombre+email +pass(≥ 6)	201 + token, rol turista	✓
TC-02	Registro email duplicado	Registrar email ya existente	409 "dato duplicado"	✓
TC-03	Registro contraseña corta	pass de 3 caracteres	400 con mensaje claro	✓
TC-04	Login válido	email+pass correctos	200 + token JWT	✓
TC-05	Login inválido	pass incorrecta	401 "credenciales incorrectas"	✓

TC-06	Listar experiencias	GET /experiencias	200 + lista con rating y guía	✓
TC-07	Filtros de búsqueda	? categoria=pesc a&precio_max=100000	solo resultados que cumplen	✓
TC-08	Reservar con cupos	reservar 2 de 8 cupos	201 reserva "pendiente"	✓
TC-09	Reservar sin cupos	reservar más del disponible	409 "no hay cupos"	✓
TC-10	Pago Stripe	checkout → tarjeta 4242...	reserva "confirmada", pago "completado"	✓
TC-11	Reseña 1–5	publicar reseña con estrellas	aparece en tiempo real	✓
TC-12	Reseña fuera de rango	puntuación = 7	400 rechazado	✓
TC-13	Solicitud de guía	turista llena formulario	aparece en panel admin	✓
TC-14	Aprobación de guía	admin aprueba solicitud	el usuario pasa a rol guía	✓
TC-15	Crear experiencia (guía)	guía publica con punto en mapa	201 + Product/Price en Stripe	✓

3. Pruebas de seguridad

ID	Verificación	Método	Resultado
SEC-01	Contraseñas encriptadas	Revisar columna password_hash (bcrypt)	✓ No hay texto plano

SEC-02	Acceso sin token	Llamar endpoint protegido sin JWT	✓ 401
SEC-03	Acceso con rol incorrecto	Turista intenta crear experiencia	✓ 403
SEC-04	No auto-asignarse rol guía	Registrarse pidiendo rol: admin/guia	✓ Siempre queda turista
SEC-05	No pagar reserva ajena	Pagar reserva de otro usuario	✓ 403
SEC-06	Claves fuera del repo	Revisar que .env esté en .gitignore	✓ No se sube
SEC-07	Inyección SQL	Enviar 'OR 1=1 en campos	✓ Consultas parametrizadas la bloquean

4. Matriz de cobertura

Módulo	Casos	Cubierto
Autenticación	TC-01...05, SEC-01...04	✓ 100%
Experiencias	TC-06, 07, 15	✓
Reservas	TC-08, 09	✓
Pagos	TC-10, SEC-05	✓
Reseñas	TC-11, 12	✓
Verificación de guía	TC-13, 14	✓
Seguridad transversal	SEC-01...07	✓

5. Reporte de bugs (ejemplos reales del proyecto)

ID	Severidad	Descripción	Estado	Solución
BUG-01	● Alta	El registro permitía auto-asignarse rol <code>guía</code> (riesgo de seguridad)	Cerrado	El backend ahora fuerza <code>turista</code> ; el rol <code>guía</code> solo lo da el admin
BUG-02	● Media	Sin webhook local, el pago no confirmaba la reserva	Cerrado	Se agregó verificación de sesión en el <code>success_url</code> (confirmación doble)
BUG-03	● Baja	Texto poco visible en modo oscuro en algunas vistas	Cerrado	Variantes <code>dark:</code> en componentes

Plantilla de bug:ID · Título · Severidad · Pasos para reproducir · Resultado actual · Resultado esperado · Evidencia (screenshot) · Estado.

6. Métricas de calidad

- **Casos ejecutados:** 22 (15 funcionales + 7 seguridad).
- **Tasa de éxito:** 100% tras correcciones.
- **Bugs encontrados / cerrados:** 3 / 3.
- **Cobertura de historias de usuario:** 13/13.

7. Automatización (recomendación / si aplica)

- **Backend:** Jest + Supertest para pruebas de endpoints.
- **Frontend/E2E:** Playwright o Cypress (flujo registro → reserva → pago).
- **CI/CD:** GitHub Actions que corra los tests en cada Pull Request antes de fusionar.
- **Cobertura:** reporte con c8/istanbul.

```
# .github/workflows/test.yml (propuesta) name: tests on:
[pull_request] jobs:  test:      runs-on: ubuntu-latest
steps:                - uses: actions/checkout@v4          - uses:
actions/setup-node@v4      with: { node-version: 20 }      -
run: cd backend && npm ci && npm test
```

8. Herramientas a demostrar

- **Postman / curl** para pruebas de API.
- **GitHub Pull Requests** para Code Review.
- **Tabla de casos y reporte de bugs** (este documento).



Guion de presentación (3-5 min)

1. **(30s)** "Soy Jhon, encargado de QA; aseguro que la plataforma sea confiable y segura."
2. **(2 min)** Muestro la **tabla de casos de prueba** y ejecuto 2-3 en vivo (login inválido, reservar sin cupos, pagar reserva ajena → 403).
3. **(1 min)** Reporte de bugs encontrados y cómo se resolvieron con el equipo (PRs).
4. **(30s)** Preguntas: pruebas de seguridad y cobertura.